Name bhavya rana
Sapid 590034689
Batch 15

Algorithm

1. Start.
2. Read the size of the array, `size`.
3. Check that `size` is positive.
4. Declare the variable-length integer array `nums[size]`.
5. Read `size` elements into `nums[]`.
6. Pass the array address and the address of `size` to `majority_element()`.
7. Inside `majority_element()`:
   - For each element, initialize `count = 0`.
   - Compare it with every element of the array.
   - Increment `count` whenever a matching element is found.
   - If `count > size / 2`, display that element as the majority element and return.
8. If no element satisfies the condition, display `-1`.
9. Stop.

C Program

```
#include <stdio.h>

void majority_element(int *nums, int *size)
{
    int i, j, count;

    for (i = 0; i < *size; i++)
    {
        count = 0;

        for (j = 0; j < *size; j++)
        {
            if (*(nums + i) == *(nums + j))
            {
                count++;
```

```
            }
        }

        if (count > (*size / 2))
        {
            printf("Majority element: %d\n", *(nums + i));
            return;
        }
    }

    printf("-1\n");
}

int main()
{
    int size, i;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid size.\n");
        return 0;
    }

    int nums[size];

    printf("Enter %d elements: ", size);

    for (i = 0; i < size; i++)
    {
        scanf("%d", &nums[i]);
    }

    /* Passing array and size using their addresses */
    majority_element(nums, &size);

    return 0;
}
```

Test Case Table

Test	Size	Array	⌊size/2⌋	Expected Output

1	5	3 3 2 3 3	2	Majority element: 3
2	7	2 2 1 1 1 2 2	3	Majority element: 2
3	6	1 2 3 2 3 3	3	-1
4	1	5	0	Majority element: 5
5	4	1 1 2 2	2	-1
6	6	7 7 7 7 2 3	3	Majority element: 7

Execution

